

HighFrame: Uma Ferramenta para Desenvolvimento em Alto Nível e Deployment Automático de Sistemas Distribuídos Baseados em Componentes

Saulo Eduardo Galilleo Souza dos Santos^{1,2}, Artêmio Oliveira de Andrade Junior¹, Victor Machado Pasqualino¹, Tarcísio da Rocha¹, Felipe Oliveira Carvalho¹

¹Departamento de Computação – Universidade Federal de Sergipe (UFS)
Caixa Postal 353 – 49100-000 – São Cristóvão – SE – Brasil

²Instituto Federal de Educação, Ciênc. e Tec. de Sergipe (IFS) - Campus São Cristóvão
Caixa Postal 11 – 49100-000 – São Cristóvão – SE – Brasil

{saulogalilleo,victorpasqualino}@yahoo.com.br

{tarcisiorocha,ocfelipe}@gmail.com, artemiojr182@hotmail.com

Abstract. *This paper presents HighFrame, an integrated solution for high-level development and deployment that aims to reduce the complexity of developing heterogeneous component-based distributed systems. The developer keeps the focus on the business of the system (generic components in POJO style + Fraclet annotations) and uses a graph model to define the distributed system architecture; HighFrame performs the deployment process of the system automatically, generating technical code of the component models and the remote bindings, distributing, instantiating and making the the system ready for use.*

Resumo. *Este trabalho apresenta o HighFrame, uma solução integrada para desenvolvimento e deployment em alto nível que tem como propósito diminuir a complexidade do desenvolvimento de sistemas distribuídos heterogêneos baseados em componentes. O desenvolvedor mantém o foco no desenvolvimento do negócio do sistema (componentes genéricos tipo POJO + anotações Fraclet) e usa um modelo gráfico para definir a arquitetura distribuída do sistema; o HighFrame realiza o processo de implantação do sistema de forma automática, gerando o código técnico dos modelos componentes e dos bindings remotos, distribuindo e instanciando o sistema, deixando-o pronto para o uso.*

1. Descrição e motivação do problema tratado pela ferramenta

Os sistemas distribuídos tem se mostrado cada vez mais complexos sendo compostos por partes heterogêneas interconectadas dinamicamente na formação de sistemas ainda mais ricos[Blair et al. 2011]. Esse tem sido um dos grandes desafios da nova geração de projetistas de middlewares e outras soluções para o desenvolvimento de sistemas distribuídos.

Uma abordagem promissora que tem sido adotada no desenvolvimento de sistemas distribuídos complexos é a engenharia de software baseada em componentes. O uso de um modelo de componentes permite o desenvolvimento de componentes reutilizáveis, que expõe operações de dependência (serviços requeridos e fornecidos), promovendo também a modularidade e configurabilidade, o que facilita a construção de sistemas

dinâmicos. Atualmente, diversos modelos de componentes para sistemas distribuídos já disponibilizam recursos para reconfiguração de *middlewares* e sistemas distribuídos (ex: Fractal, SCA, OSGi) [Bennour et al. 2009]. Porém, apesar desses benefícios, a adoção deste modelo de desenvolvimento pode representar a introdução de mais complexidade no processo de desenvolvimento de *software* uma vez que o desenvolvedor precisa destinar esforços para o desenvolvimento de código técnico do modelo de componentes específico que foi adotado na implementação da aplicação [Rouvoy and Merle 2009].

Além desse desafio, o processo de desenvolvimento de sistemas distribuídos baseados em componentes envolve outros obstáculos como: (i) a comunicação remota entre os componentes distribuídos; (ii) a implantação dos componentes em nós distribuídos; e (iii) a interconexão de componentes desenvolvidos em modelos heterogêneos. Essas dificuldades podem tornar complexa a tarefa de desenvolver um sistema distribuído, mesmo quando esse possui requisitos funcionais simples. Dado esse nicho emergente dos sistemas distribuídos baseados em componentes, um dos desafios relevantes é o de diminuir a complexidade do desenvolvimento desses tipos de sistemas distribuídos.

Neste trabalho é apresentada uma ferramenta gráfica para planejamento e deployment em alto nível de arquiteturas de sistemas distribuídos baseados em componentes – a HighFrame. Ela é uma solução integrada que reduz a complexidade do desenvolvimento desses tipos de sistemas, oferecendo soluções que lidam diretamente com os seus obstáculos. Com ela o desenvolvedor volta a preocupação para o desenvolvimento dos requisitos funcionais do sistema e da sua arquitetura em um modelo gráfico de alto nível. Outras preocupações complexas envolvidas são tratadas automaticamente pela HighFrame como, (i) a criação do código técnico dos modelos de componentes (ex: OpenCom, Fractal) e do código técnico da comunicação remota entre os componentes distribuídos (ex: Web Services, RMI), (ii) a implantação dos componentes nos nós distribuídos e (iii) a interconexão entre componentes heterogêneos (com modelos de componentes diferentes).

Algumas ferramentas publicadas na literatura também identificam a necessidade de soluções no sentido de facilitar o processo de desenvolvimento de sistemas baseados em componentes. Por exemplo, [Sentilles et al. 2009] apresenta uma solução para o desenvolvimento de sistemas embarcados com geração automática de componentes específicos no modelo SaveCCM; [Riba and Cervantes 2007] apresentam uma ferramenta MDA para a construção de aplicações orientadas a serviços baseadas em componentes onde modelos de alto nível são transformados em componentes específicos no modelo OSGi. Porém, além destes trabalhos se limitarem apenas a uma tecnologia de componentes específica, eles não consideram os problemas dos cenários que envolvem distribuição. Além disso, esses trabalhos não retiram do desenvolvedor a complexidade de desenvolvimento de código técnico para comunicação remota, como também não provê comunicação remota entre componentes heterogêneos.

2. HighFrame

A HighFrame tem como objetivo simplificar o desenvolvimento de sistemas distribuídos baseados em componentes, permitindo ao desenvolvedor manter o foco no negócio do sistema. Ela propõe uma solução integrada que inclui: (i) desenvolvimento de componentes baseado em implementações genéricas focada no negócio do sistema; (ii) definição

da arquitetura do sistema baseado em um modelo gráfico de alto nível; (iii) *deployment* automático da arquitetura nos nós distribuídos disponibilizando o sistema na sua forma funcional. A Arquitetura do HighFrame framework é apresentada na figura 1.

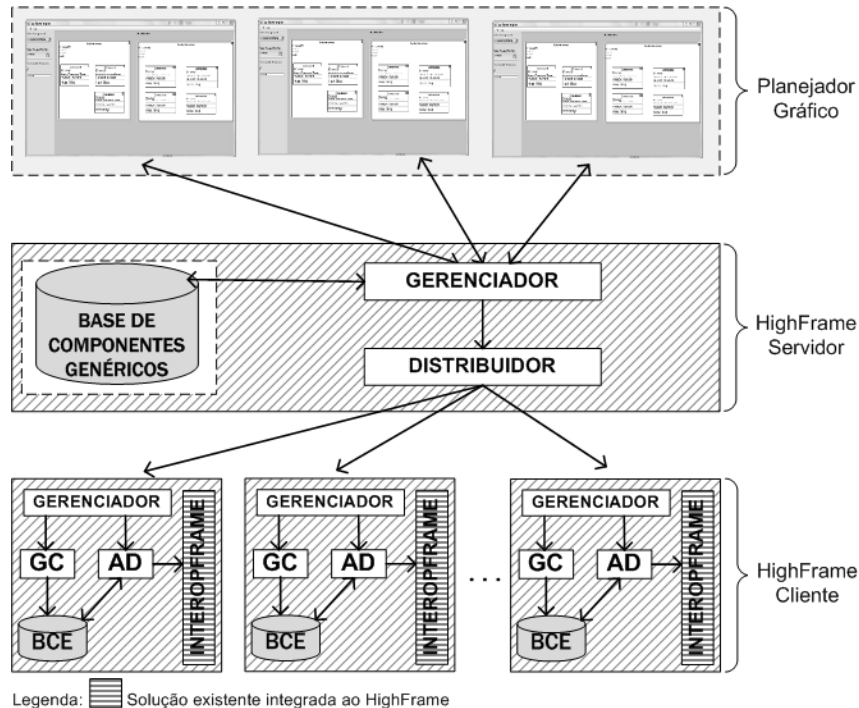


Figura 1. Arquitetura do HighFrame framework

Para que o desenvolvedor possa manter o foco no negócio do sistema a ser desenvolvido, a HighFrame fornece uma camada de abstração para o desenvolvimento de componentes. Esta camada é baseada no modelo de anotações Fraclet [Rouvoy and Merle 2009], que representa preocupações do desenvolvimento baseado em componentes independente de tecnologia de componentes. Por exemplo, desenvolver um componente Java com esse modelo seria equivalente a desenvolver uma classe no estilo POJO incluindo as anotações Fraclet no código fonte como: *@Component* (marca um componente), *@Interface* (marca uma interface do negócio do sistema), *@Requires* (marca uma interface requerida) e *@Provides* (marca uma interface provida). O código fonte anotado pode representar um componente e passa a ser inserido em uma base de componentes genéricos do *HighFrame*, esta base centraliza todos componentes genéricos em um servidor para que sejam disponibilizados para reutilização em qualquer nova arquitetura independente de um dado modelo de componentes específico, distribuição ou formas de comunicação.

O HighFrame Servidor disponibiliza através do módulo gerenciador um catálogo de componentes genéricos existentes na base de componentes genéricos, isto para que o desenvolvedor possa compor a arquitetura do sistema e o plano de *deployment* distribuído do sistema. O processo de composição de um sistema pode ser feito graficamente com a HighFrame Designer (planejador gráfico) – com ele o desenvolvedor, define os nós distribuídos, arrasta os componentes para os respectivos nós e cria as interconexões entre os mesmos. Além disso, o desenvolvedor também define qual(uais) modelo(s) de componentes específico(s) a serem usado(s) no *deployment* e as formas de *binding* remoto entre

eles (ex: Web Services SOAP/REST, RMI...). Para representar internamente a arquitetura projetada pelo desenvolvedor, a HighFrame usa uma ADL específica chamada HighADL. Um plano de *deployment* é também criado automaticamente com a HighADL que é posteriormente usado pela HighFrame para a realização do *deployment* distribuído. A arquitetura e o plano de *deployment* em HighADL são submetidos através do HighFrame Designer para o HighFrame Servidor. O módulo gerenciador do HighFrame Servidor processa as ADL's e seleciona cada componente genérico referenciado para realizar a distribuição para os seus respectivos nós distribuídos.

Em cada nó distribuído reside o HighFrame Cliente que é o responsável por receber o plano de *deployment* e os componentes genéricos para realizar *deployment* da subarquitetura do nó (um projeto pode envolver vários nós, cada um com sua subarquitetura). Depois disso, o HighFrame Cliente usa o seu Gerador de Componentes (GC) para gerar os componentes específicos em um dado modelo (ex: OSGi) com base no componente genérico e armazená-los na Base de Componentes Específicos (BCE). Em seguida é acionado o Agente de *Deployment* (AD) que faz a instalação, ativação e interconexão (*binding* local) dos componentes como também realiza a interligação (*binding* remoto) entre os componentes que se conectam a componentes de outros nós. Para o processo de *binding* remoto é utilizada uma solução chamada InteropFrame [Nascimento et al. 2013] que foi desenvolvida para prover interconexão automática entre componentes distribuídos possivelmente heterogêneos. O InteropFrame gera automaticamente os *proxies* que realizam o *binding* remoto. A Arquitetura foi implementada baseada em plugins, de forma a ser extensível para novos modelos de componentes através de *templates*.

3. HighFrame Designer

O HighFrame Designer é o planejador gráfico que pertence ao HighFrame. Ele tem o objetivo de fornecer um ambiente para composição da arquitetura de sistemas distribuídos e criação de plano de *deployment* baseado em um modelo gráfico com as facilidades do modo “arrastar e soltar”. O HighFrame Designer possibilita que o usuário desenvolvedor crie a arquitetura do sistema distribuído através de um catálogo de componentes genéricos. A ferramenta disponibiliza ao usuário desenvolvedor os componentes genéricos para utilização no modo *design* da ferramenta. Estes componentes genéricos são desenvolvidos previamente independentemente de tecnologia de componentes, como descrito anteriormente. Com isso, ele retira do desenvolvedor a necessidade de conhecer o código técnico para desenvolvimento de componentes, comunicação remota distribuída e *deployment* distribuído. Ao invés disso, o usuário foca no planejamento da arquitetura e na definição dos parâmetros para o *deployment* do sistema como, o modelo de componentes específico a ser usado, os tipos de *binding* remoto e os endereços dos nós distribuídos.

3.1. Funcionamento

A figura 2 apresenta um exemplo no HighFrame Designer de uma arquitetura composta por dois componentes, cada um em um nó distinto. Na caixa “Select Component” do ambiente de design estão disponíveis os componentes genéricos que foram armazenados no HighFrame Servidor. O HighFrame Designer permite criação de uma arquitetura completa como também de uma subarquitetura independente. Subarquiteturas independentes podem ser criadas e salvas para posterior reutilização. Elas ficam disponíveis na ferramenta na caixa “Select SubArchitecture”.

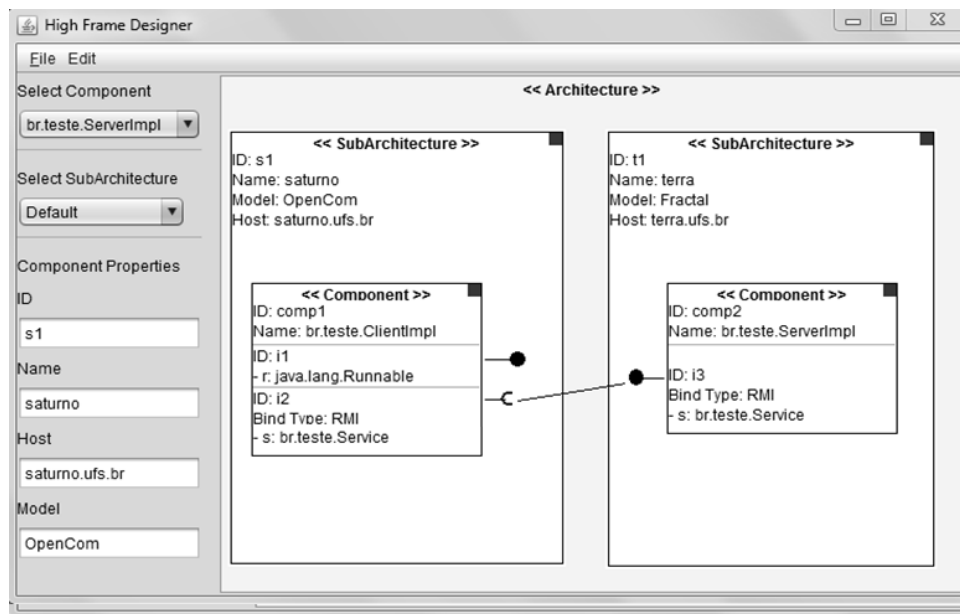


Figura 2. Ambiente da IDE

Uma subarquitetura refere-se a um subsistema que irá ser executado em um nó distribuído. A subarquitetura pode ser vista como um componente composto que exporta interfaces para conexão com outras subarquiteturas distribuídas. No exemplo apresentado temos duas subarquiteturas, uma denominada “saturno”, que possui o endereço saturno.ufs.br como nó de destino. A outra é denominada “terra”, que possui como nó de destino terra.ufs.br. Na subarquitetura é definido também o modelo de componentes para o qual os componentes presentes na mesma serão transformados. Cada nó distribuído deverá conter o HighFrame Cliente, pois este módulo distribuído irá proporcionar o suporte ao processo de *deployment* e à comunicação entre componentes distribuídos.

O uso do HighFrame Designer como ferramenta para composição do sistema e criação do plano de *deployment* tem como produto final dois arquivos para submissão ao HighFrame Servidor. O Planejador possui uma função chamada “Execute Deployment”, esta função faz a transformação do modelo gráfico em alto nível para dois arquivos XML em HighADL: *data.xml* e *plan.xml* que consistem na representação da arquitetura distribuída e no plano de *deployment*, respectivamente. O código fonte 1 apresenta a HighADL gerada para o exemplo apresentado na figura 2.

Código Fonte 1. HighADL - arquivo data.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<architecture>
  <subarchitecture idsubarchitecture="s1">
    <component id="comp1" name="clientImpl">
      <provideinterface idinterface="i1" name="r" signature="java.lang.Runnable"/>
      <requiredinterface idinterface="i2" name="s" signature="Service"/>
    </component>
    <exportinterface idinterface="i2"/>
  </subarchitecture>
  <subarchitecture idsubarchitecture="t1">
    <component id="comp2" name="serverImpl">
      <provideinterface idinterface="i3" name="s" signature="Service"/>
    </component>
    <exportinterface idinterface="i3"/>
  </subarchitecture>
</architecture>
```

```
</architecture>
```

Na HighADL são definidas as subarquitecturas que compõem o sistema e quais interfaces são exportadas. Este XML refere-se a descrição da arquitectura do sistema baseado em componentes. O segundo arquivo é o plano de *deployment*, que possui informações necessárias para que o HighFrame execute o *deployment* distribuído. O quadro 2 apresenta o plano de *deployment* gerado para o exemplo apresentado.

Código Fonte 2. Plano de deployment - arquivo plan.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<plan>
  <deployment idsubarchitecture="s1" componentmodel="OpenCom" host="saturno.ufs.br"/>
  <deployment idsubarchitecture="t1" componentmodel="Fractal" host="terra.ufs.br"/>
  <remotebindind idexportinterfaceclient="i2" idexportinterfaceclient="i3" typebinding="
    RMI"/>
</plan>
```

O plano de *deployment* contém informações para que o HighFrame execute o *deployment* distribuído. No código apresentado, temos a definição de *deployment* das duas arquitecturas correspondentes. A primeira com a subarquitectura ‘s1’ destinada ao nó saturno.ufs.br com a transformação dos seus componentes genéricos em componentes do modelo OpenCom. A segunda, ‘t1’, destinada ao nó terra.ufs.br com a transformação dos componentes genéricos em componentes do modelo Fractal. O elemento *remotebinding* define as interfaces exportadas de cada subarquitectura destinada a nós distintos e também o tipo de conexão entre os nós distribuídos. Para o exemplo acima, a comunicação remota definida foi RMI.

Apesar de, por motivo de espaço, o exemplo apresentado acima ter sido o mais trivial possível, a HighFrame é flexível o suficiente para permitir a definição uma arquitectura com várias subarquitecturas, cada uma delas com vários componentes interconectados. Cada subarquitectura pode usar um modelo de componentes diferente, se for o caso. Isso é útil tanto pelo fato de permitir a reutilização de componentes já existentes quanto pelo fato de permitir que o desenvolvedor explore as diferentes potencialidades e características de cada modelo de componentes na composição de subarquitecturas diferentes.

Com a HighFrame é possível também que as subarquitecturas de uma mesma arquitectura possam ser interconectadas usando *bindings* remotos diferentes. Essa flexibilidade permite também explorar a melhor forma de *binding* para cada caso. Por exemplo, pode-se escolher RMI como *binding* entre componentes de duas subarquitecturas localizadas em nós de uma rede local para favorecer o desempenho da comunicação, enquanto que usar WebServices REST para o binding entre duas subarquitecturas localizadas em redes distintas pelo fato do RMI não oferecer suporte padrão para redes distintas. A escolha é feita pelo desenvolvedor, porém a geração do código técnico do *binding* é feita automaticamente pelo HighFrame através do InteropFrame.

4. Demonstração planejada para o Salão de Ferramentas-SBRC

A demonstração que foi preparada para o SBRC usará um cenário simples com dois nós (que podem ser dois notebooks pessoais pré-configurados em uma rede local básica). O primeiro nó conterá uma instância do HighFrame Server e uma instância do HighFrame-Client; o segundo nó conterá uma instância do HighFrame Client e uma instância do

HighFrame Designer (apesar de que essas instâncias podem ser totalmente distribuídas em nós distintos). Ela consistirá no processo de planejamento e implantação de um servidor web distribuído chamado Comanche WebServer [Rouvoy and Merle 2009]. A arquitetura baseada em componentes do Comanche WebServer a ser usada é apresentada na figura 3. Ela é distribuída em dois nós: o “No1” com o *Frontend* e o “No2” com o *Backend* do exemplo.

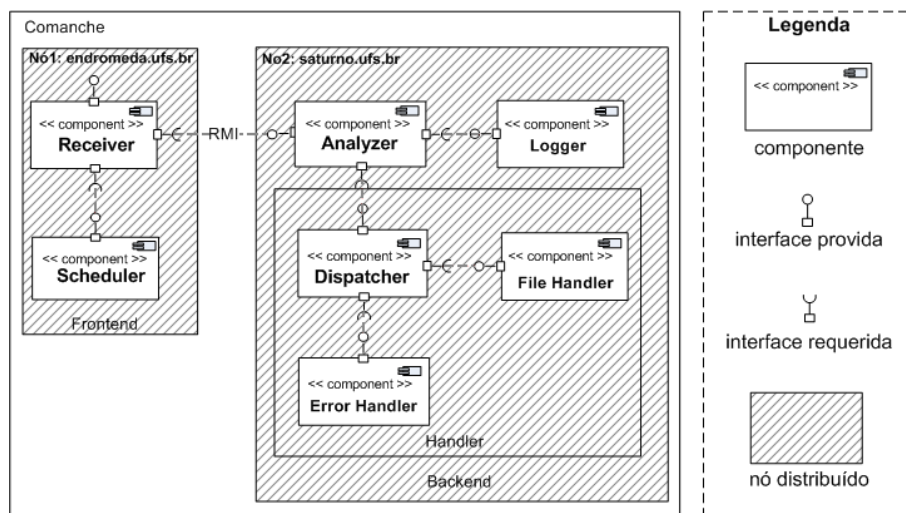


Figura 3. Modelo do comanche WebServer

Em particular, o servidor web comanche é composto pelos seguintes componentes: *Receiver*, *Scheduler*, *Analyzer*, *Logger*, *Dispatcher*, *FileHandler* e *ErrorHandler*. Mais detalhes sobre o papel de cada um desses componentes podem ser encontrados na documentação do HighFrame disponibilizada no link abaixo. Para o modelo proposto, teremos transformações para o modelo OpenCom no *Frontend* e para o modelo Fractal no *Backend*. O componente *Receiver* localizado no nó “endromeda.ufs.br” conecta-se através de RMI ao componente *Analyzer* localizado no nó “saturno.ufs.br” ligando assim o *Frontend* ao *Backend*.

A demonstração será iniciada mostrando brevemente como um componente genérico é desenvolvido com o foco único do negócio do sistema. Para isso mostraremos trechos do componente “*Analyzer*” destacando a sua característica POJO e a simplicidade do uso das anotações Fraclet. Em seguida será mostrado como a arquitetura do Comanche Web Server pode ser criada de forma rápida e simples com o uso do HighFrame Designer. A próxima etapa, será o processo de *deployment* do Web Server mostrando como o HighFrame realiza automaticamente o *deployment* do *Frontend* no “No1” (primeiro notebook) e do *Backend* no “No2” (segundo notebook) usando somente os componentes genéricos e o modelo em alto nível criado no planejador. Por fim, será realizado um teste simples de funcionamento com o Servidor Web distribuído que, neste momento, já terá sido ativado e já estará pronto para ser requisitado.

Apesar da arquitetura desse exemplo ser composta por vários componentes, todo esse processo de demonstração levará poucos minutos, enfatizando a capacidade da ferramenta de agilizar o processo de desenvolvimento e *deployment* de sistemas distribuídos baseados em componentes, considerando que os componentes genéricos da composição

já estarão implementados e armazenados na base do HighFrame.

O **HighFrame**, o seu **manual de uso** e sua **documentação** estão disponíveis em <https://github.com/saulogalilleo/HighFrame/wiki/HighFrame>

5. Conclusão e Trabalhos Futuros

Neste artigo apresentamos o HighFrame, uma solução integrada para o desenvolvimento e *deployment* em alto nível de sistemas distribuídos baseados em componentes genéricos visando a redução da complexidade do desenvolvimento dos mesmos. Suas principais características: (i) permite ao desenvolvedor manter o foco no negócio do sistema (implementação dos requisitos funcionais na forma de componentes genéricos) e especificação da arquitetura distribuída através de um modelo gráfico de alto nível; (ii) automatização do processo de *deployment* envolvendo a geração do código técnico dos componentes específicos, distribuição dos componentes entre os nós distribuídos, instanciação e interligação entre os mesmos; (iii) geração automática dos *proxies* para realização do *binding* remoto. Um outro ponto relevante é que a arquitetura do HighFrame é flexível para dar suporte a novos modelos de componentes ou novas formas de *bindings* remotos na forma de *plugins*, ou seja, sem precisar alterar ou recompilar as partes existentes ferramenta.

Os trabalhos futuros incluem: (i) o desenvolvimento de plugins para outros modelos de componentes como o CCM e o SCA e novas formas de *binding* como WebServices REST (atualmente são suportados os modelos de componentes OpenCom e Fractal e os *bindings* do tipo RMI e WebServices SOAP); (ii) suporte a componentes desenvolvidos em outras linguagens (atualmente somente componentes em Java são suportados); (iii) desenvolvimento de uma versão do planejador integrada à IDE Eclipse.

References

- Bennour, B., Henrio, L., and Rivera, M. (2009). A reconfiguration framework for distributed components. In *Proceedings of the 2009 ESEC/FSE Workshop on Software Integration and Evolution Runtime*, SINTER'09, pages 49–56, NY, USA. ACM.
- Blair, G., Paolucci, M., Grace, P., and Georgantas, N. (2011). Interoperability in Complex Distributed Systems. In *11th International School on Formal Methods for the Design of Computer, Communication and Software Systems: Connectors for Eternal Networked Software Systems*. Springer.
- Nascimento, S. C., Carvalho, F. O., and da Rocha, T. (2013). Towards interoperability between heterogeneous distributed components. In *12th International Workshop on Adaptive and Reflective Middleware*, ARM '13, pages 3:1–3:7, NY, USA. ACM.
- Riba, N. and Cervantes, H. (2007). A mda tool for the development of service-oriented component-based applications. In *ENC*, pages 149–156. IEEE Computer Society.
- Rouvoy, R. and Merle, P. (2009). Leveraging component-based software engineering with fractal. *annals of telecommunications*, 64:65–79.
- Sentilles, S., Pettersson, A., Nystrom, D., Nolte, T., Pettersson, P., and Crnkovic, I. (2009). Save-ide - a tool for design, analysis and implementation of component-based embedded systems. In *Proceedings of the 31st International Conference on Software Engineering*, ICSE '09, pages 607–610, Washington, USA. IEEE.